



Programação WEB Avançada

Universidade de Vila Velha

Comprometida com a excelência no ensino, pesquisa e inovação.

✉ wanderson.santana@uvv.br

3

Arquitetura com Application Factory e Contexto no Flask

Resumo

Nesta unidade aprofundamos a arquitetura de aplicações Flask, dando continuidade à organização modular introduzida anteriormente. O foco está na compreensão dos principais problemas estruturais de projetos web em crescimento, especialmente importações circulares, acoplamento excessivo e má gestão do ciclo de vida da aplicação. A partir disso, apresentamos o padrão *Application Factory*, o uso correto de extensões desacopladas, e os conceitos fundamentais de *Application Context* e *Request Context*. Também discutimos a organização em pacotes, inicialização preguiçosa (lazy initialization), e boas práticas para projetos escaláveis, testáveis e manuteníveis. Ao final, o estudante será capaz de estruturar aplicações Flask profissionais seguindo princípios de engenharia de software e arquitetura limpa.

*“Arquitetura de software é sobre tomar decisões que permitam evolução
sem colapso.”*

Adaptado de Martin Fowler

Problemas Estruturais em Aplicações Flask

À medida que aplicações web evoluem, a estrutura inicial — geralmente simples e funcional — tende a se tornar insuficiente, especialmente em sistemas que crescem em complexidade e modularização [1, 2]. Em projetos Flask iniciantes, é comum concentrar toda a lógica em um único arquivo, o que funciona bem para protótipos, mas rapidamente gera dificuldades de manutenção, escalabilidade e testabilidade.

Dois problemas principais surgem com frequência:

- Importações circulares entre módulos;
- Acoplamento excessivo entre a aplicação, extensões e rotas.

Esses problemas aparecem, sobretudo, quando a aplicação cresce e passa a ser dividida em múltiplos módulos, como `models`, `routes`, `services` e `extensions`.

O Problema das Importações Circulares

Importações circulares ocorrem quando dois ou mais módulos dependem mutuamente entre si durante o carregamento do interpretador. Em Flask, isso acontece frequentemente quando o objeto `app` é criado em um módulo principal e importado diretamente em diversos outros módulos.

Esse tipo de dependência cíclica é um problema conhecido em sistemas modulares e pode comprometer o carregamento correto de módulos em linguagens interpretadas como Python [3, 4].

Por exemplo:

- O arquivo `app.py` cria a aplicação;
- O módulo de rotas importa `app`;
- O módulo principal importa as rotas.

Esse ciclo pode impedir a inicialização correta do projeto, resultando em erros difíceis de diagnosticar e comportamentos inesperados.

O código a seguir irá introduzir o processo de criação de um aplicativo mínimo a partir do Flask. Lembrem-se que a classe `Flask` é a base do nosso framework. É através dela que criamos a instância do nosso app, configuramos a aplicação e acessamos a maioria das funcionalidades. O que não vem da classe `Flask` provém das extensões que podemos instalar e/ou desenvolver.

```
1 from flask import Flask
2 app = Flask(__name__)
3 @app.route("/")
4 def index():
5     return "Ola Mundo!"
6 @app.route("/contato")
7 def contato():
8     return "<form><input type='text'></form>"
```

Um aplicativo básico é geralmente chamado de app, e passamos a variável mágica do Python, `__name__`, para a criação da instância.

As linhas 1 e 2 do código acima, basicamente resumem conceitualmente um aplicativo Flask. O que vamos adicionar a esse aplicativo são as views. Para isso, utilizamos o decorador `@app.route`, que define a rota. Este decorador intercepta e gerencia as rotas, associando uma função que criamos à URL correspondente. No caso, a rota `"/"` será a URL base do nosso projeto, e a função `index` retornará "Olá Mundo!".

Essa é a nossa aplicação Flask simples. Para executá-la, abra um terminal e use o comando `flask run`. Em seguida, acesse a URL¹ `localhost:5000` no navegador, e você verá a mensagem "Olá Mundo!".

E, até aqui, nenhuma novidade!

No entanto, em um projeto real, provavelmente adicionaremos mais rotas. Além da rota principal, podemos criar uma nova rota chamada `contato`, que servirá para um formulário onde o usuário poderá enviar informações de contato. A função `contato` retornará um HTML simples.

¹Em particular, sou fã do endereço `127.0.0.1:5000` — *there's no place like this!*

Assim, ao acessarmos a URL `localhost:5000/contato` no navegador, seremos apresentados a um formulário com um campo de entrada. Precisamos incluir botões e outros elementos que geralmente fazem parte de um formulário. Porém, a aplicação não se limitará a esse formulário de contato.

Essas funções que associamos às rotas são chamadas de `views` porque são responsáveis por enviar informações para a camada de visualização. Diferente de outros frameworks, como Django, que seguem padrões como MVC (Model View Controller) ou MVT (Model View Template), o Flask não impõe um padrão de projeto específico. Portanto, temos a liberdade de criar nossa própria estrutura.

À medida que o projeto avança, uma prática comum é dividir a aplicação em múltiplos arquivos. Por exemplo, podemos criar um arquivo chamado `views.py` para organizar nossas `views` separadamente do arquivo principal `app.py`. No entanto, se você fizer isso sem as devidas importações, ao rodar o Flask, você receberá um erro informando que as URLs não foram encontradas.

Quando executamos `flask run`, o Flask procura pelo arquivo `app.py` e o executa linha a linha. Este arquivo é o ponto de entrada da aplicação. Se o `app` não estiver acessível, as `views` em `views.py` não serão reconhecidas.

Para resolver isso, precisamos importar as `views` no arquivo principal. Por exemplo, podemos usar:

Código: `app.py`

```
1 from flask import Flask
2 from views import index, contato
3
4 app = Flask(__name__)
```

No entanto, se tentarmos rodar dessa forma, **aqui surgirá o famoso erro de importação circular**. Ele ocorrerá porque, ao importar as `views`, o Flask tentará executar o arquivo `views.py`, que por sua vez tenta importar o `app`. Essa situação gera um ciclo onde cada arquivo depende do outro para ser inicializado, resultando em um *deadlock*.

Quando você vê uma mensagem de erro indicando que o aplicativo foi parcialmente inici-

alizado, isso geralmente aponta para um erro de importação circular. Esse problema pode ocorrer em qualquer programa Python que tente dividir o código em múltiplos arquivos. O Python segue uma ordem de importação e, ao tentar resolver as dependências, ele pode entrar em um *loop* infinito.

Código: views.py

```
1 from app import app
2 @app.route("/")
3 def index():
4     return "Ola Mundo!"
5 @app.route("/contato")
6 def contato():
7     return "<form><input type='text'></form>"
```

Para entender melhor, vamos analisar o que acontece durante a execução:

- 1. **Importação do app:** o Flask inicia a execução do arquivo `app.py`, importando a classe Flask e criando a instância.
- 2. **Importação das Views:** ao tentar importar as views, o Flask executa o arquivo `views.py` linha a linha.
- 3. **Tentativa de Acesso ao app:** quando o `views.py` tenta usar o decorador `@app.route`, ele não consegue encontrar a instância `app` porque a execução ainda não retornou ao `app.py`.

A Figura (3.1) ilustra a descrição acima.

No `app.py`	E então no `views.py`
<pre>from flask import Flask from views import index app = Flask(__name__)</pre> 	<pre>from app import app @app.route('/') def index(): return "Hello World"</pre>

Figura 3.1: Resumo do nosso exemplo de *Circular import*.

Essa situação também resulta numa classificação de erro conhecida como “`ImportError`”, indicando que o aplicativo foi parcialmente inicializado.

Para evitar isso, poderíamos pensar em usar o padrão de importação em que o app é criado antes das views serem importadas, garantindo que o Flask tenha o contexto necessário para associar as rotas corretamente. *“Mas, aí...”*

A “Gambiarra” do Import no Final do Arquivo

Diante do problema de importação circular, uma solução bastante comum — embora não seja considerada elegante — consiste em adiar a importação das views para o final do arquivo principal. Na prática, muitos desenvolvedores² movem a linha de importação para depois da criação da instância da aplicação:

Código: app.py

```
1 from flask import Flask
2 app = Flask(__name__)
3
4 from views import index, contato
```

E no arquivo de views mantemos tudo igual:

Código: views.py

```
1 from app import app
2 @app.route("/")
3 def index():
4     return "Ola Mundo!"
5 @app.route("/contato")
6 def contato():
7     return "<form><input type='text'></form>"
```

À primeira vista, essa abordagem parece resolver o problema de importação circular. E, de fato, em muitos casos ela funciona. Mas por quê? Vamos entender isso, falando sobre a **ordem de execução do sistema de imports do Python**.

²Não vamos pré-julgar tais desenvolvedores. Entretanto, precisamos registrar que bons desenvolvedores Python têm conhecimento legítimo sobre a arquitetura de seus códigos e das boas práticas extensivamente recomendadas na PEP 8 – Style Guide for Python Code.

O Python executa os arquivos de cima para baixo, de forma sequencial. Assim, quando o interpretador executa o arquivo `app.py`, ocorre a seguinte sequência:

1. O Python importa o módulo `flask`;
2. A instância `app = Flask(__name__)` é criada;
3. Somente depois disso ocorre a importação de `index` e contato do módulo `views`.

Nesse momento, quando o arquivo `views.py` é executado e tenta fazer:

```
1 from app import app
```

o objeto `app` já existe no contexto de execução, pois a instância foi criada anteriormente. Dessa forma, o ciclo de dependência não causa mais falha imediata de inicialização, permitindo que a aplicação funcione.

Por que Essa Solução Não é Recomendada? Apesar de funcional, essa técnica viola uma das principais recomendações da PEP 8, o guia oficial de estilo do Python, que orienta que as importações devem ser organizadas no topo do arquivo.

Colocar *imports* no final do arquivo ou dentro de funções:

- reduz a legibilidade do código;
- dificulta a manutenção;
- torna a estrutura do programa menos previsível;
- pode introduzir comportamentos inesperados em projetos maiores.

Em comunidades *Pythonistas*, a organização e a clareza do código são altamente valorizadas. Assim, `imports` fora do topo do arquivo costumam ser vistos como um indicativo de problema estrutural na arquitetura do projeto, e não como uma solução definitiva.

Além disso, outra prática que aparece com frequência é a realização de `imports` dentro de funções, como forma de evitar o ciclo de importação. Embora isso funcione tecnicamente, também não é considerado uma boa prática em termos de design de software.

Lazy Imports e Execução Tardia (Lazy Evaluation)

Para compreender uma solução mais elegante, é necessário entender um conceito fundamental: o sistema de `import` do Python é imediato. Ou seja, quando utilizamos:

```
1 from modulo import objeto
```

o Python tenta resolver essa importação naquele exato momento da execução.

Logo, uma forma mais adequada de lidar com dependências circulares é **adiar** a execução de determinadas partes do código. Esse comportamento é conhecido como execução preguiçosa (*lazy execution*) — também denominada posterior.

Em programação, a principal estrutura que permite **execução posterior** é o conceito de objetos *callable*. No Python, são considerados *callables*:

- Funções;
- Métodos;
- Classes;
- Qualquer objeto que possa ser invocado com **parênteses**.

Ao definirmos uma função, estamos descrevendo um comportamento que só será executado posteriormente, quando ela for chamada. Isso nos permite controlar melhor o momento em que determinadas dependências serão carregadas.

Transição para uma Solução Arquitetural Elegante — a “gambiarra” do `import` no final do arquivo funciona porque altera a ordem de execução, mas não resolve a causa raiz do problema: o acoplamento excessivo entre módulos e a inicialização rígida da aplicação.

Em projetos Flask de médio e grande porte, precisamos de uma abordagem mais limpa, escalável e alinhada com boas práticas de engenharia de software. Em vez de depender da ordem dos imports, a solução recomendada é **estruturar a aplicação de modo que sua criação ocorra de forma controlada e tardia**.

Essa abordagem é implementada por meio de um padrão arquitetural conhecido como *Application Factory*, no qual a aplicação Flask deixa de ser uma variável global e passa a ser criada por uma **função** responsável pela sua configuração e inicialização.

Então, vamos em frente com essa técnica!

Aplicando o Padrão Application Factory de Forma Simples

Para evitar importações circulares e melhorar a arquitetura do sistema, o próprio ecossistema do Flask recomenda o uso do padrão conhecido como *Application Factory* [5].

Até aqui, vimos que mover o import para o final do arquivo resolve o problema de importação circular, mas não é uma solução elegante. Precisamos de uma abordagem que elimine o acoplamento estrutural entre os módulos.

A proposta do Flask para resolver esse tipo de problema é o uso do padrão conhecido como *Application Factory*.

A ideia central é que, em vez de criar a aplicação diretamente no escopo global:

```
1 app = Flask(__name__)
```

vamos encapsular sua criação dentro de uma função. Essa função será responsável por construir e configurar a aplicação somente quando for chamada.

Isso altera completamente o comportamento do sistema: nada acontece no momento da importação do módulo. A aplicação só passa a existir quando a função for invocada.

Portanto, precisamos reorganizar nosso exemplo anterior em dois arquivos.

Código: app.py

```
1 import views
2 from flask import Flask
3
4 def create_app():
5     """Factory principal"""
6     app = Flask(__name__)
```

```
7 views.init_app(app)
8 return app
```

Observe alguns pontos importantes:

- não existe mais `app = Flask(__name__)` no escopo global;
- a aplicação é criada apenas dentro da função `create_app()`;
- o módulo `views` é importado normalmente no topo do arquivo;
- o objeto `app` é passado como argumento para outro módulo.

Esse comportamento caracteriza uma forma de inicialização preguiçosa (*lazy initialization*), técnica amplamente utilizada para controle do ciclo de vida de objetos e redução de acoplamento em sistemas complexos [6, 1].

Agora vejamos o segundo arquivo.

Código: views.py

```
1 """Extensao Flask"""
2 def init_app(app):
3     """Inicializacao de extensoes"""
4     @app.route("/")
5     def index():
6         return "Ola Mundo!"
7     @app.route("/contato")
8     def contato():
9         return "<form><input type='text'></form>"
```

E, aqui está a mudança conceitual mais importante:

- Não existe mais `from app import app`;
- O módulo não depende diretamente do objeto global;
- A função `init_app` apenas descreve o que deve ser feito quando receber uma aplicação.

A redução do acoplamento entre módulos é um princípio central da engenharia de software e contribui diretamente para a manutenibilidade e testabilidade do sistema [2].

Por Que Isso Resolve o Circular Import? Quando o Python importa o módulo `views`, ele apenas define a função `init_app`. Nenhuma rota é registrada ainda.

Somente quando `create_app()` é executada é que:

1. A instância da aplicação é criada;
2. A função `views.init_app(app)` é chamada;
3. As rotas são registradas.

Ou seja, eliminamos a dependência circular porque:

- `views.py` não precisa importar `app`;
- `app.py` não depende da execução interna de `views`;
- A ligação entre os módulos acontece de forma controlada.

Mudança de Mentalidade

Antes, tínhamos um objeto global rígido, criado imediatamente.

Agora, temos uma função que constrói a aplicação sob demanda.

Essa diferença parece pequena no código, mas é enorme do ponto de vista arquitetural. Passamos de um modelo acoplado para um modelo configurável e extensível.

Mais do que resolver um erro técnico, o padrão *Application Factory* nos ensina a estruturar aplicações Flask de forma previsível, modular e alinhada com boas práticas de engenharia de software.

Benefícios Arquiteturais

Com essa abordagem:

- eliminamos importações circulares;
- reduzimos acoplamento;
- melhoramos a organização modular;
- facilitamos testes automatizados;
- permitimos múltiplas instâncias da aplicação com configurações diferentes.

Mais do que resolver um erro técnico, a Application Factory representa uma mudança de mentalidade: a aplicação deixa de ser um objeto global rígido e passa a ser um componente configurável e extensível.

Flask Lifetime — Context

Um outro problema comum ao interagir com aplicações Flask está relacionado à compreensão do ciclo de vida da aplicação e de seus contextos internos, conceitos formalmente definidos na documentação do framework e em sua arquitetura baseada em WSGI [7, 8].

Dominar o Flask está diretamente relacionado ao entendimento dos três momentos fundamentais em que a aplicação pode se encontrar, que são:

- fase de configuração (setup da aplicação);
- contexto de aplicação (*Application Context*);
- contexto de requisição (*Request Context*).

Compreender essas camadas é essencial para evitar erros como acesso indevido a variáveis globais, uso incorreto de objetos como `current_app`, erros do tipo “*Working outside of application context*”, além de problemas no uso de extensões.

Quando criamos uma instância Flask:

ou quando essa criação ocorre dentro de uma *Application Factory*, entramos no que podemos chamar de fase de configuração.

Nesse momento:

- a aplicação está sendo construída;
- ainda não existe uma requisição ativa;
- nenhum contexto de `request` foi iniciado;
- estamos apenas montando a estrutura do sistema.

Essa é a fase apropriada para compor a aplicação.

O que pode ser feito durante a configuração?

- Adicionar configurações:

```
1 @app.route('/')

```

ou

```
1 app.add_url_rule('/', ...)
```

- Registrar blueprints:

```
1 app.register_blueprint(foo_bar)
```

- Inicializar extensões:

```
1 admin = Admin(app)
```

ou no padrão moderno:

```
1 admin.init_app(app)
```

- Registrar hooks³:

```
1 @app.before_request
2 def before():
3     pass
4
5 @app.errorhandler(404)
6 def not_found(error):
7     return "Nao encontrado", 404
```

- Compor o sistema via factories secundárias:

```
1 views.init_app(app)
2 extensions.init_app(app)
```

Essa fase é extremamente importante porque define toda a estrutura da aplicação antes que qualquer requisição seja processada.

Application Context

O *Application Context* representa o estado da aplicação ativa.

Ele permite acessar informações globais da aplicação sem precisar importar diretamente o objeto `app`. Para isso, o Flask utiliza proxies⁴ como:

³Hooks são pontos de extensão no ciclo de vida da requisição que permitem executar funções antes, durante ou após o processamento de uma requisição HTTP, sendo parte fundamental do mecanismo de middleware interno do Flask [7].

⁴Quando dizemos “o Flask utiliza proxies como `current_app`, `request`, `g`”, estamos falando de proxies locais de contexto (*context-local proxies*), e não de proxies de rede. Eles são objetos especiais que funcionam como “representantes dinâmicos” de outro objeto real, que depende do contexto atual (aplicação ou requisição). Ou seja, o `current_app` não é o `app` diretamente, o `request` não é o objeto de requisição glo-

- `current_app`
- `g`

Para isso, o Flask utiliza proxies de contexto (*context-local proxies*), como `current_app`, `request` e `g`, que fornecem acesso dinâmico ao estado da aplicação e da requisição ativa [9, 7].

Por exemplo:

```
1 from flask import current_app
2
3 def exemplo():
4     valor = current_app.config['FOO']
```

O objeto `current_app` só funciona quando há um contexto de aplicação ativo. Caso contrário, o Flask lançará um erro indicando que estamos “trabalhando fora do contexto da aplicação”.

O contexto de aplicação é criado automaticamente quando:

- uma requisição é iniciada;
- um comando CLI do Flask é executado;
- o contexto é ativado manualmente.

Também podemos ativá-lo manualmente:

```
1 with app.app_context():
2     # código que precisa de contexto
```

Request Context

O *Request Context* é criado a cada requisição HTTP. Ele contém informações específicas daquela requisição, como:

`request`, `g` não é um objeto global tradicional. Na verdade, eles são wrappers que apontam para o objeto correto em tempo de execução, dependendo do contexto ativo.

- cabeçalhos HTTP;
- parâmetros de URL;
- dados de formulário;
- sessão;
- objeto request.

Exemplo:

```
1 from flask import request
2
3 @app.route("/hello")
4 def hello():
5     nome = request.args.get("nome")
6     return f"Ola {nome}"
```

Assim como o `current_app`, o objeto `request` só pode ser utilizado quando há uma requisição ativa. Fora desse contexto, ocorrerá erro.

Relação Entre os Contextos

É importante observar que:

- Todo *Request Context* possui um *Application Context*;
- Nem todo *Application Context* possui um *Request Context*.

Durante uma requisição HTTP:

1. O Flask cria o contexto de aplicação;
2. Em seguida cria o contexto de requisição;
3. Executa a view;
4. Finaliza e remove os contextos.

Conclusão Conceitual

Grande parte dos erros avançados no Flask ocorre por confusão entre:

- Fase de configuração (setup);
- Contexto de aplicação;
- Contexto de requisição.

Entender claramente quando cada um desses momentos existe é fundamental para utilizar extensões corretamente, evitar importações indevidas, trabalhar com CLI, implementar testes automatizados e projetar aplicações maiores.

Síntese da Unidade

Nesta unidade, avançamos da construção inicial de aplicações Flask para uma abordagem arquitetural mais madura, baseada no padrão Application Factory e na correta utilização dos contextos internos do framework. Compreendemos como evitar importações circulares, estruturar projetos modulares e inicializar extensões de forma desacoplada.

Esse conjunto de conceitos estabelece a base necessária para o desenvolvimento de aplicações web profissionais, preparadas para testes automatizados, múltiplos ambientes de execução e evolução contínua do software.

Lista de Exercícios

3.1 Objetivo principal do Application Factory no Flask

Considere a seguinte estrutura:

```
1 from flask import Flask
2
3 def create_app():
4     app = Flask(__name__)
5     return app
```

Do ponto de vista arquitetural, qual é o principal objetivo do uso do padrão Application Factory em aplicações Flask?

- (A) Melhorar a performance do servidor HTTP
- (B) Evitar a criação de rotas dinâmicas
- (C) Reduzir acoplamento e evitar importações circulares
- (D) Substituir o uso de Blueprints
- (E) Eliminar a necessidade de configuração

3.2 Importação circular em aplicações Flask

Considere os arquivos: **Código: app.py**

```
1 from flask import Flask
2 from views import init_app
3
4 app = Flask(__name__)
5 init_app(app)
```

Código: views.py

```
1 from app import app
2
3 @app.route("/")
4 def index():
5     return "Ola"
```

Qual é o problema estrutural mais provável dessa arquitetura?

- (A) Sobrecarga de memória
- (B) Falha na renderização de templates
- (C) Erro de tipagem dinâmica
- (D) Incompatibilidade com WSGI
- (E) Dependência circular entre módulos

3.3 Importação circular em aplicações Flask

Considere os arquivos: **Código: app.py**

```
1 from flask import Flask
2 from views import init_app
3
4 app = Flask(__name__)
5 init_app(app)
```

Código: views.py

```
1 from app import app
2
3 @app.route("/")
4 def index():
5     return "Ola"
```

Qual é o problema estrutural mais provável dessa arquitetura?

- (A) Sobrecarga de memória
- (B) Dependência circular entre módulos
- (C) Falha na renderização de templates
- (D) Erro de tipagem dinâmica
- (E) Incompatibilidade com WSGI

3.4 Lazy initialization na Application Factory

Analise o código:

```
1 def create_app():  
2     app = Flask(__name__)  
3     return app
```

Por que dizemos que esse padrão utiliza inicialização preguiçosa (*lazy initialization*)?

- (A) Porque a aplicação é criada apenas quando a função é invocada
- (B) Porque o Flask executa todas as rotas antecipadamente
- (C) Porque as extensões são removidas da aplicação
- (D) Porque o Python adia a compilação do código
- (E) Porque o servidor executa em modo assíncrono

3.5 Regra arquitetural sobre o objeto app

Em projetos Flask estruturados com factories, qual prática é considerada uma boa regra de arquitetura?

- (A) Importar o objeto app em todos os módulos
- (B) Declarar o app como variável global obrigatória
- (C) Criar múltiplos objetos Flask globais
- (D) Evitar importar diretamente o objeto app nos módulos da aplicação
- (E) Substituir o app por variáveis de ambiente

3.6 Função do init_app em módulos Flask

Considere:

```
1 def init_app(app):  
2     @app.route("/")  
3     def index():  
4         return "Ola Mundo!"
```

Qual é a principal função do padrão `init_app(app)`?

- (A) Criar múltiplas instâncias do Flask

- (B) Registrar componentes na aplicação de forma desacoplada
- (C) Executar requisições HTTP automaticamente
- (D) Substituir o uso de decorators
- (E) Configurar o servidor WSGI

3.7 Contexto de aplicação (Application Context)

Considere o uso de:

```
1 from flask import current_app
2 valor = current_app.config["DEBUG"]
```

Esse código só funciona corretamente quando:

- (A) O servidor está em modo debug
- (B) O banco de dados está conectado
- (C) O Flask está rodando com Gunicorn
- (D) Existe um contexto de aplicação ativo
- (E) O arquivo config.py foi importado

3.8 Request Context no ciclo de vida do Flask

Qual das opções descreve corretamente o *Request Context*?

- (A) Estado global permanente da aplicação
- (B) Contexto criado apenas durante a compilação
- (C) Contexto criado a cada requisição HTTP
- (D) Contexto exclusivo do banco de dados
- (E) Substituto do Application Factory

3.9 Proxies de contexto no Flask

Objetos como `request`, `current_app` e `g` são classificados como:

- (A) Variáveis globais estáticas

- (B) Middlewares HTTP
- (C) Decoradores de rota
- (D) Objetos WSGI diretos
- (E) Proxies de contexto (context-local proxies)

3.10 Fase de configuração da aplicação Flask

Durante a fase de configuração (setup) de uma aplicação Flask, qual ação é considerada adequada?

- (A) Acessar dados do objeto request
- (B) Processar formulários HTTP
- (C) Registrar blueprints e extensões
- (D) Manipular sessões de usuário
- (E) Executar queries de requisição ativa

3.11 Relação entre Application Context e Request Context

Sobre a relação entre os contextos do Flask, assinale a alternativa correta:

- (A) Todo Application Context possui obrigatoriamente um Request Context
- (B) Todo Request Context possui um Application Context associado
- (C) Ambos são criados apenas na inicialização do servidor
- (D) O Request Context existe antes do Application Context
- (E) Eles são independentes e não se relacionam

3.12 Execução do comando flask run com Application Factory

Considere que o projeto possui apenas:

```
1 def create_app():  
2     app = Flask(__name__)  
3     return app
```

Por convenção, o que o comando `flask run` fará ao encontrar essa estrutura?

- (A) Executar automaticamente a função `create_app`
- (B) Criar um servidor sem contexto
- (C) Ignorar a aplicação por não existir variável global `app`
- (D) Exigir obrigatoriamente um arquivo `run.py`
- (E) Compilar a aplicação antes de executar

Referências Bibliográficas

- [1] FOWLER, M. *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley, 2002.
- [2] MARTIN, R. C. *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River: Prentice Hall, 2009. ISBN 9780132350884.
- [3] LUTZ, M. *Learning Python*. 5. ed. Sebastopol: O'Reilly Media, 2013.
- [4] ROSSUM, G. V.; FOUNDATION, P. S. *Python Language Reference*. 2024. <https://docs.python.org>. Acesso em: 2026.
- [5] Pallets Projects. *Application Factories — Flask Patterns*. 2024. <https://flask.palletsprojects.com/en/latest/patterns/appfactories/>. Acesso em: 2026.
- [6] GAMMA, E. et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley, 1994.
- [7] Pallets Projects. *Flask Documentation*. 2024. <https://flask.palletsprojects.com>. Acesso em: 2026.
- [8] RONACHER, A. *Flask Design and Architecture Notes*. 2018. <https://lucumr.pocoo.org>. Criador do Flask.
- [9] Pallets Projects. *Werkzeug Context Locals Documentation*. 2024. <https://werkzeug.palletsprojects.com>. Acesso em: 2026.